Informe Técnico

1. Resumen de Arquitectura del Sistema El proyecto "Bank Legacy Batch" moderniza un sistema bancario tradicional migrándolo hacia una arquitectura de microservicios orientada a eventos. El sistema se apoya en contenedores Docker y se orquesta para ofrecer alta disponibilidad, seguridad y procesamiento asíncrono. La entrada principal de la aplicación opera como un patrón BFF (Backend For Frontend), diseñado para centralizar y asegurar las peticiones provenientes de distintos canales (como ATM).

2. Componentes del Sistema y Puertos

- **BFF y Spring Batch (`bank-legacy-batch` - Puerto 8443):** Microservicio principal que recibe tráfico HTTPS mediante un certificado `keystore.p12`. Expone los endpoints de negocio, valida autorizaciones estrictas mediante JWT y encabezados personalizados (`X-Operacion`, `X-Canal`), y ejecuta procesos de negocio y por lotes (Spring Batch).
- **Eureka Server (`eureka-server` - Puerto 8761):** Servidor de descubrimiento de servicios que permite a los microservicios registrarse y encontrarse dinámicamente sin acoplar direcciones IP fijas.
- **Config Server (`config-server` - Puerto 8888):** Servidor centralizado que provee las configuraciones externas a los microservicios al momento de su arranque.
- **Kafka y Zookeeper (Puertos 9092 y 2181):** Clúster de mensajería asíncrona encargado de recibir los eventos generados por las transacciones bancarias, desacoplando el procesamiento y mejorando la escalabilidad del sistema.
- **MySQL (`mysql-db` - Puerto 3306):** Motor de base de datos relacional para el almacenamiento persistente de las cuentas, saldos y metadatos de Spring Batch.

3. Diagrama y Flujo de Integración

1. El cliente (Postman/Front-end) envía una petición cifrada al puerto **8443**.
2. La clase `BffSecurityService` intercepta la petición, validando la existencia e integridad del token de autorización y haciendo un match exacto de los encabezados de negocio esperados (ej. `X-Operacion`).
3. Si la validación es exitosa, el BFF consulta el saldo en **MySQL**.
4. La arquitectura contempla la inyección del patrón **Circuit Breaker** (Resilience4j) para evitar caídas en cascada si la base de datos o Kafka demoran en responder.
5. Eventos paralelos y procesos por lotes se delegan al broker de **Kafka**, coordinado por **Zookeeper**, completando el ciclo asíncrono.